

GraphMind: From Operational Traces to Self-Evolving Workflow Automation

Yiwen Zhu
Microsoft, USA
yiwzh@microsoft.com

Joyce Cahoon
Microsoft, USA
jcahoon@microsoft.com

Anna Pavlenko
Microsoft, USA
anna.pavlenko@microsoft.com

Qiushi Bai
Microsoft, USA
qiushibai@microsoft.com

Nima Shahbazi*
University of Illinois
Chicago, USA
nshahb3@uic.edu

Divya Vermareddy
Microsoft, USA
dvermareddy@microsoft.com

Meina Wang
Microsoft, USA
meiwa@microsoft.com

Mathieu Demarne
Microsoft, USA
mdemarne@microsoft.com

Swati Bararia
Microsoft, USA
barariaswati@microsoft.com

Wenjing Wang
Microsoft, USA
wenjwang@microsoft.com

Hemkesh Vijaya
Kumar
Microsoft, USA
hvijayakumar@microsoft.com

Hannah Lerner
Microsoft, USA
hannahlerner@microsoft.com

Katherine Lin
Microsoft, USA
katlin@microsoft.com

Steve Toscano
Microsoft, USA
stoscano@microsoft.com

Miso Cilimdzc
Microsoft, USA
misoc@microsoft.com

Subru Krishnan
Microsoft, Spain
subru@microsoft.com

ABSTRACT

Complex operational workflows coordinating personnel, tools, and information are central to enterprise operations, yet end-to-end automation remains challenging due to extensive requirements for human inputs and the inability to adapt over time. We present GraphMind, an end-to-end system that constructs, executes, and evolves action-centric workflow graphs without human effort. The system operates in three phases. First, a *scalable offline pipeline* extracts structured workflow graphs from large volumes of human resolution traces, capturing problems, actions, and their causal relationships. Second, an *online multi-agent traversal* engine navigates the graph to dynamically construct and execute workflows, combining graph-guided retrieval with LLM-driven reasoning at each step. Third, *Adaptive Traversal Reinforcement* (ATR) reinforces successful traversal paths and decays stale elements. This closed-loop mechanism enables the graph to self-optimize and adapt to shifting operational conditions. GraphMind has been deployed across four production cloud database services for incident investigation. Evaluated on production data, the system substantially outperforms a Trace-RAG baseline in mitigation reach, groundedness, and diagnostic throughput, scoring 4.95/5 in blind expert review. The ATR layer provides further gains across all metrics, demonstrating that workflow graphs can learn and improve from execution-derived feedback.

1 INTRODUCTION

Organizations rely on complex operational workflows to coordinate personnel, tools, and information across domains like customer support, service operations, and change management. While linear tasks are easily scripted, end-to-end automation remains an open

challenge due to knowledge-intensive processes spanning heterogeneous tools and requiring iterative, context-dependent decisions. Consider a diagnostic workflow in a database system:

[PROBLEM] *ActivateWorkspace failing at 95.7% in region* → **[ACTION]** *Query telemetry to analyze failure timeline* → **[OBSERVATION]** *Sustained failures over 14+ hours, ruling out transient spike* → **[ACTION]** *Drill into exception details for failed operations* → **[OBSERVATION]** *No exception messages; suggests infrastructure-level outage* → ... → **[ACTION (MITIGATION)]** *Restart system services*

Although LLM-based agents [5, 23, 26, 35, 39] and agent skills (e.g., markdown-based instruction files in tools such as Claude Code [2] and GitHub Copilot CLI [16]) have renewed interest in workflow automation, purely prompt-driven approaches remain brittle in enterprise environments. Current practice relies on manually authored knowledge artifacts that are (i) *costly to author*, demanding deep domain expertise; (ii) *difficult to maintain*, as production systems evolve while artifacts become stale; (iii) *inefficient at retrieval time*, consuming excessive context tokens; and (iv) *unable to capture the long tail* of edge cases that experts handle through improvisation but never document. Meanwhile, operational traces—historical records of how experts actually carried out complex workflows—are abundant yet remain underutilized as structured, reusable knowledge.

We argue that **action-centric workflow graphs**, constructed from operational traces, offer a far more efficient and agent-friendly format than unstructured playbooks. Moreover, these graphs can learn from their own execution: each traversal generates signal about which paths succeeded, which were dead ends, and which novel sequences the agent discovered. We draw inspiration from **Ant Colony Optimization (ACO)** [12, 13]: ants collectively discover short paths by depositing pheromone on traversed routes;

*Work done during an internship at Microsoft.

Table 1: Comparison of knowledge formats for workflow automation.

Aspect	Playbooks-RAG	Agent Skills	Trace-RAG ¹	GraphMind
Storage efficiency	✗ Redundant	✗ Per-skill	✗ Per-incident	✓ Shared nodes
Agent friendliness	✗ Verbose/Noisy	~ Structured	~ Condensed	✓ Graph traversal
Human effort	High	High	None	None
Coverage	Fixed top- <i>k</i>	Fixed per-skill	Fixed top- <i>k</i>	Grows with traces
Self-evolution	✗	✗	✗	✓ Via ATR

successful paths accumulate stronger pheromone while unsuccessful paths evaporate. We formalize this analogy as **Adaptive Traversal Reinforcement (ATR)**, a mechanism that allows the workflow graph to self-optimize by depositing reinforcement along successful paths and decaying stale elements. As Table 1 summarizes, this design uniquely combines storage efficiency through shared graph nodes, agent-friendly traversal, zero human effort, expandable coverage, and continuous self-evolution—advantages that no existing knowledge format offers jointly.

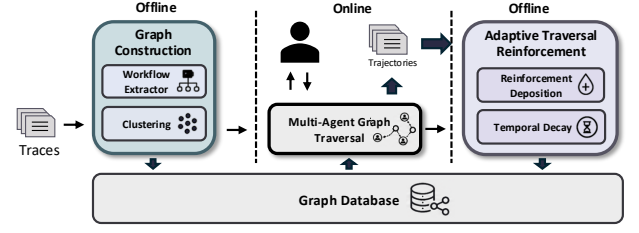
Introduction to GraphMind. We present GraphMind, a system realizing this vision through three core pillars (see Figure 1).

Pillar 1: Offline graph construction (warm start). As LLM context windows expand and agentic retrieval matures [3, 28], the central challenge has shifted from maximizing retrieval recall to navigating a three-way trade-off among **cost**, **latency**, and **accuracy**. GraphMind occupies a pragmatic middle ground between naive long-context ingestion of raw traces, which inflates online cost and latency, and fully resolved entity-centric knowledge graphs [14, 17], which incur heavy offline construction cost. Our offline pipeline extracts **actions** as the central unit of a lightweight **workflow graph** with a fixed, minimal schema, reducing online retrieval to a bounded graph traversal (Section 3).

Pillar 2: Online multi-agent traversal. Rather than single-step retrieval, the online phase performs *iterative graph traversal*: at each step the agent retrieves a relevant subgraph, reasons over context and execution history, and selects the most promising action to **exploit**; if no suitable action exists, it pivots to a different graph region to **explore**. Each traversal generates a complete *trajectory* that serves as the raw material for adaptive traversal reinforcement (Section 4).

Pillar 3: Adaptive traversal reinforcement. Inspired by Ant Colony Optimization (ACO), we treat accumulated trajectories as reinforcement signals in a closed-loop feedback framework. *Reinforcement deposition* strengthens edges and nodes along successful paths, while *temporal decay* fades stale elements. Over time, the graph self-organizes: frequently successful paths become highways guiding future agents, while ineffective paths fade—encoding collective decision-making experience without explicit knowledge engineering (Section 5).

Application to incident management. We instantiate this paradigm for incident management in large-scale cloud database

**Figure 1: Three-phase architecture of GraphMind: offline construction provides the warm-start graph, online traversal generates trajectories, and adaptive traversal reinforcement continuously refines the graph.**

services at Microsoft, where GraphMind has been deployed in production across four services. Incident management involves high-stakes decisions and heterogeneous telemetry, providing a rigorous stress test for self-evolving workflow automation.

Contribution. We present GraphMind, to our knowledge the first system that automatically constructs, executes, and evolves action-centric workflow graphs from operational traces in a closed loop, requiring zero manual authoring. A scalable offline pipeline extracts lightweight workflow graphs using a fixed, minimal schema; a multi-agent online engine traverses and executes them with LLM-driven reasoning; and Adaptive Traversal Reinforcement (ATR), a closed-loop mechanism inspired by Ant Colony Optimization, enables the graph to self-optimize from its own execution experience. Deployed across four production cloud database services, GraphMind outperforms a Trace-RAG baseline in mitigation reach, groundedness, and expert-rated quality, demonstrating that workflow graphs can continuously learn and improve from autonomous agent execution at scale.

2 RELATED WORK

ACO and swarm intelligence. ACO was introduced for combinatorial optimization [12, 13] and extended in variants such as ACS and MAX-MIN [11, 30], with applications to routing [10], scheduling [21], and feature selection [31]. GraphMind adopts the same reinforcement-and-decay intuition but applies it to evolving workflow graphs for LLM-guided automation.

Process mining. Classical process mining recovers workflows from event logs [32, 33]; GraphMind targets noisier natural-language traces and uses LLM extraction to build an executable workflow graph.

Knowledge graphs and graph-enhanced retrieval. Prior work combines LLMs with knowledge graphs through fine-tuning or retrieval [24, 37], while GraphRAG methods use graph structure to improve retrieval [17, 20, 25]. KG refinement via link prediction [4, 8, 36] learns static representations; LLMs can also generate structured queries for grounded context [19, 27]. GraphMind keeps the graph external, retrieves iteratively, and lets execution outcomes reshape it.

AIOps and incident management. D-Bot, Panda, and RCACopilot apply LLMs to troubleshooting [6, 29, 41]; TRIANGLE and

¹Trace-RAG uses LLM-condensed summaries of operational traces as retrieval units; while this eliminates manual authoring and improves readability over raw logs, the resulting documents remain per-incident, lacking cross-incident deduplication, explicit causal structure, and self-evolution.

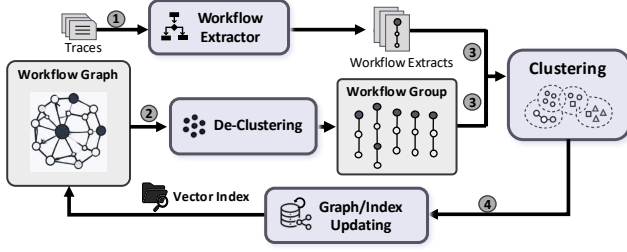


Figure 2: Incremental graph construction. As new operational traces are generated continuously in large volume, the pipeline extracts structured workflow from each batch, merges them with the existing workflow graph after de-clustering, and applies semantic clustering to deduplicate nodes, producing an updated, consolidated workflow graph without full reconstruction.

FLASH study triage [38, 40, 42]. Neuro-symbolic approaches use structured graphs to constrain LLM outputs [7]. GraphMind complements this line by automatically learning and improving an action-centric knowledge base.

LLM agents. Tool-augmented agents and agentic RAG support iterative reasoning [3, 26, 28, 39]. GraphMind grounds that loop in a learned workflow graph with reinforcement-weighted priors.

3 PILLAR 1: OFFLINE GRAPH CONSTRUCTION (WARM START)

This section describes the offline pipeline that transforms raw operational traces into a structured, queryable workflow graph at scale. In the ACO analogy, this phase corresponds to constructing the initial search space before any traversals have occurred. In production, traces grow continuously in large volume (e.g., as new incidents are resolved manually), producing an ever-expanding source of expert decision-making data. The pipeline therefore supports *incremental construction*, integrating newly extracted knowledge without rebuilding the graph. As illustrated in Figure 2, construction proceeds in four steps. (1) *Workflow Extraction*: each incoming batch of traces is processed by an LLM-based extractor that produces one workflow extract per trace (a subgraph), capturing the problems encountered, actions taken, and their causal ordering (Section 3.2). (2) *De-clustering and Merging*: the current version of the workflow graph is de-clustered, reverting clustered nodes back to their raw workflow extracts, which are then merged with the newly extracted ones. (3) *Clustering*: semantically similar or identical nodes across all workflow extracts are identified and merged, deduplicating the graph while preserving distinct nodes (Section 3.3). (4) *Graph and Index Update*: the updated graph is synchronized to the production database and the vector index is updated to enable efficient search and retrieval during online traversal (Section 3.4).

3.1 Graph Schema

We define a typed graph schema with three entity types and four edge relations. **Domain ($\mathcal{D}$)**: a bounded operational area sharing common systems, tools, and ownership (e.g., payments operations,

infrastructure reliability). **Problem ($\mathcal{P}$)**: an operational issue requiring investigation or resolution within a domain (e.g., degraded performance, authorization failures). **Action ($\mathcal{A}$)**: an executable step, observational or interventive, taken to gather evidence or resolve a problem.

Four edge types capture semantic connections: **CAUSES** ($p_i \rightarrow p_j$, problem causation), **RESOLVES** ($a \rightarrow p$, action addresses problem), **LEADS_TO** ($a_i \rightarrow a_j$, sequential workflow dependency), and **BELONGS_TO** (p or $a \rightarrow d$, domain classification). This yields a workflow graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where $\mathcal{V} = \mathcal{D} \cup \mathcal{P} \cup \mathcal{A}$ and $\mathcal{E}$ comprises the typed edges, supporting structured traversal from root causes to resolution strategies.

3.2 Workflow Extractor

The system employs a **multi-agent extraction pipeline** that preserves causal and temporal relationships. It ingests heterogeneous sources including issue-tracking databases and historical resolution traces, proceeding in three stages. *Stage 1: Data Preprocessing* normalizes formats via document parsing, converts embedded images/tables to text via vision-language models, and segments documents using domain-aware heuristics. *Stage 2: Workflow Construction* extracts problem statements, filters for actionable content, identifies discrete action steps with preconditions, and assembles reasoning chains conforming to the graph schema. *Stage 3: Pruning and Alignment* removes redundant nodes and verifies type consistency.

3.3 Clustering

As workflows accumulate, the graph acquires semantically similar nodes describing the same problem or action in different terms. After de-clustering and merging with existing workflows (see Section 3.4), we apply a lightweight clustering procedure per node type using off-the-shelf embedding models for easy maintenance and incremental updates. While a more sophisticated approach could leverage LLMs to perform semantic deduplication with richer contextual understanding, such methods do not scale to production-sized graphs containing tens of thousands of nodes due to prohibitive cost and latency. We therefore opt for the simplest effective approach: embedding-based agglomerative clustering, which achieves comparable quality at a fraction of the cost (Section 6.1). The clustering pipeline proceeds as follows: (1) **Input canonicalization**: node text is canonicalized via regex masking to remove trace-specific tokens (GUIDs, timestamps, IPs). (2) **Embedding**: canonicalized descriptions are embedded using a pre-trained model (e.g., text-embedding-3-large). (3) **Agglomerative clustering**: nodes are grouped using agglomerative clustering [18] with cosine distance threshold τ tuned per node type. (4) **Representative node generation**: for each cluster, three members (longest, shortest, median description) are selected as representatives n_C^* , inheriting the union of all member edges. (5) **Edge rewiring**: parent and successor edges of each member are redirected to n_C^* ; original members are disabled but retained for future incremental updates.

3.4 De-Clustering, Graph Update and Maintenance

When new traces arrive, the existing cluster representative nodes are removed and all original members are re-enabled (de-clustering). The raw workflows are then merged with the newly extracted ones, and clustering is re-applied to produce the updated graph. This approach maintains a clean, deduplicated graph while supporting incremental updates without full reconstruction.

4 PILLAR 2: ONLINE MULTI-AGENT GRAPH TRAVERSAL

Each online traversal corresponds to a single autonomous execution: a team of agents iteratively navigates the workflow graph, selecting edges and nodes based on a combination of reinforcement weights (encoding historical reward signals) and LLM-driven reasoning, executing the chosen actions, and producing a trajectory that serves as input to the ATR layer (Section 5). In our deployment, action nodes include *diagnostic actions* (e.g., KQL telemetry queries) and *mitigation actions* (e.g., restart or escalation recommendations). For safety, GraphMind autonomously executes diagnostic queries while high-impact mitigations are surfaced as recommendations unless explicitly approved by operators.

4.1 Orchestration

We leverage an existing enterprise orchestration framework [42] as the online agentic execution environment; the orchestration layer is modular and replaceable by any Model Context Protocol (MCP)-compatible client [1] (e.g., GitHub Copilot CLI [16], Claude Code [2]). The framework employs a **nested control flow**: an *outer loop* over agents and an *inner loop* over skills. Each outer iteration maintains a **global context** C_t (long-term memory); each skill writes to a **temporary context** $c_{t,j}$ (short-term memory). The Coordination Agent drives reasoning via Graph Loader and Action Planner skills, while Execution Agents (e.g., Kusto Agent) carry out actions and return results for iterative refinement.

4.2 Agentic Graph-Traversal Algorithm

The learned workflow graph can contain tens of thousands of nodes and edges, each edge carrying a reinforcement weight $\phi(e)$ that reflects accumulated traversal reward (initialized uniformly during warm start and updated by ATR in Section 5). Given this scale, the online execution engine requires a traversal algorithm that is **efficient, robust, and accurate**, so it can identify the most relevant next actions based on principled reasoning, reinforcement-weighted edge priorities, and on the outcomes of previously executed actions. Given the workflow graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ consists of typed nodes (domains, problems, actions) connected by causal and procedural edges, we organize the traversal as a nested control flow (Algorithm 1). The outer loop (index t) corresponds to major investigation iterations with context C_t ; the inner loop (index j) performs iterative graph exploration with the following tools:

Graph Loader. At each inner-loop step (t, j) , the Skill Selector dynamically determines a graph query text $s_{t,j}$ and a target node type $\tau_{t,j}$ as part of the skill invocation argument. This step requires

strong LLM reasoning: much like an MCP client [1] or function-calling agent [22] that must reason about tool semantics to determine the correct arguments, f_{LLM} must interpret the current investigation context, decide which type of graph node is most relevant, and formulate a precise query to retrieve it:

$$(s_{t,j}, \tau_{t,j}) \leftarrow f_{\text{LLM}}(C_t, c_{t,j}). \quad (1)$$

The Graph Loader then retrieves a set of top- k_p root nodes $\mathcal{R}_{t,j}$ of type $\tau_{t,j}$ using vector similarity search with minimum latency given the pre-built search index:

$$q_{t,j} \leftarrow \text{Embed}(s_{t,j}), \quad (2)$$

$$\mathcal{R}_{t,j} \leftarrow \text{Top}_{k_p}(\mathcal{V}_{\tau_{t,j}} \setminus \mathcal{R}_{t,<j}, q_{t,j}), \quad (3)$$

where $\mathcal{V}_{\tau_{t,j}} \subseteq \mathcal{V}$ is the set of nodes of type $\tau_{t,j}$, and $\mathcal{R}_{t,<j} = \bigcup_{j' < j} \mathcal{R}_{t,j'}$ denotes root nodes previously retrieved within the same outer-loop iteration t . This exclusion encourages exploration diversity by directing each re-invocation toward a different region of the graph. The loader then expands from $\mathcal{R}_{t,j}$ to extract an m -hop neighborhood subgraph via *probabilistic multi-path expansion*: at each hop, outgoing edges are sampled without replacement with probability proportional to their log-compressed reinforcement weights $\log(1 + \phi(e))$ (detailed in Section 5.4). This yields the expanded subgraph:

$$\mathcal{V}_{t,j} \leftarrow \{v \in \mathcal{V} \mid \exists r \in \mathcal{R}_{t,j}, v \text{ reached within } m \text{ hops}\}, \quad (4)$$

$$\mathcal{E}_{t,j} \leftarrow \{(u, v) \in \mathcal{E} \mid u \in \mathcal{V}_{t,j}, v \in \mathcal{V}_{t,j}\},$$

$$\mathcal{G}_{t,j} \leftarrow (\mathcal{V}_{t,j}, \mathcal{E}_{t,j}).$$

This probabilistic expansion biases the subgraph toward historically successful neighborhoods while still allowing less-traversed paths to be discovered.

Action Planner. The Action Planner selects candidate actions by conditioning an LLM on the investigation context, the extracted subgraph (rendered as a Mermaid diagram with node reinforcement weights $\phi(v)$ annotated as node properties), and system-level instructions:

$$(A_{t,j}, \Delta c_{t,j}) \leftarrow g_{\text{LLM}}(C_t, c_{t,j}, \mathcal{G}_{t,j}, \{\phi(v)\}_{v \in \mathcal{V}_{t,j}}), \quad (5)$$

where $A_{t,j}$ contains up to k_a selected actions and $\Delta c_{t,j}$ captures auxiliary reasoning outputs such as new hypotheses, observations, or suggested exploration directions. If $A_{t,j} = \emptyset$, the planner has not found actionable steps in the current graph neighborhood; in this case, $\Delta c_{t,j}$ serves as a pivoting signal containing hints about alternative hypotheses or unexplored problem areas. This context update is merged into the temporary context: $c_{t,j+1} \leftarrow c_{t,j} \cup \Delta c_{t,j}$, so that in the next inner-loop iteration the enriched $c_{t,j+1}$ guides the Graph Loader to search a different region of the graph, effectively steering exploration toward more promising neighborhoods. The planner can also reason about dependencies between actions, selecting prerequisite actions when a desired action is infeasible due to missing inputs, or decide to terminate.

Action Execution. Once the Action Planner produces a non-empty set $A_{t,j}$, the corresponding Execution Agents take over. For each selected action $a \in A_{t,j}$, a specialized agent (e.g., Kusto Agent for diagnostic queries, Mitigation Agent for remediation steps) executes the action and returns an observation $o_t^{(a)} \leftarrow \text{Exec}(a)$. The observation is then incorporated into the global investigation

Algorithm 1 Agentic Graph-Traversal with Nested Loops

Require: Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, context C_0 , params: k_p (root nodes), m (hops), k_a (actions), K (edges per hop), max iters J

```

1:  $t \leftarrow 0$ 
2: while not terminated do
3:    $j \leftarrow 0$ ;  $c_{t,0} \leftarrow \emptyset$ ;  $A_{t,j} \leftarrow \emptyset$ 
4:   while  $A_{t,j} = \emptyset$  and  $j < J$  do
5:      $(s_{t,j}, \tau_{t,j}) \leftarrow \text{fLLM}(C_t, c_{t,j})$ 
6:      $q_{t,j} \leftarrow \text{Embed}(s_{t,j})$ 
7:      $\mathcal{R}_{t,j} \leftarrow \text{Top}_{k_p}(\mathcal{V}_{\tau_{t,j}} \setminus \mathcal{R}_{t,<j}, q_{t,j})$ 
8:      $\mathcal{V}_{t,j} \leftarrow \{v \mid \exists r \in \mathcal{R}_{t,j}, \text{dist}(r, v) \leq m\}$ 
9:      $\mathcal{E}_{t,j} \leftarrow \{(u, v) \in \mathcal{E} \mid u, v \in \mathcal{V}_{t,j}\}$ 
10:     $\mathcal{G}_{t,j} \leftarrow (\mathcal{V}_{t,j}, \mathcal{E}_{t,j})$ 
11:     $(A_{t,j}, \Delta c_{t,j}) \leftarrow g_{\text{LLM}}(C_t, c_{t,j}, \mathcal{G}_{t,j})$ 
12:     $c_{t,j+1} \leftarrow c_{t,j} \cup \Delta c_{t,j}$ 
13:    if  $A_{t,j} = \emptyset$  then  $j \leftarrow j + 1$ ; continue
14:    end if
15:    break
16:  end while
17:  if  $A_{t,j} = \emptyset$  then terminate
18:  end if
19:  for all  $a \in A_{t,j}$  do
20:     $o_t^{(a)} \leftarrow \text{Exec}(a)$ ;  $C_t \leftarrow C_t \cup \{(a, o_t^{(a)})\}$ 
21:  end for
22:   $C_{t+1} \leftarrow C_t$ ;  $t \leftarrow t + 1$ 
23: end while
24: return Final diagnosis and recommended mitigation steps

```

context: $C_t \leftarrow C_t \cup \{(a, o_t^{(a)})\}$. After all actions in $A_{t,j}$ have been executed, the updated context $C_{t+1} \leftarrow C_t$ is carried forward to the next outer-loop iteration (Algorithm 1, lines 12–14), enabling subsequent iterations to reason over accumulated evidence from all prior actions. When all iterations complete and no further actions are recommended, the system reaches a termination state; an LLM then synthesizes the full accumulated context C_t into a final diagnosis report with recommended mitigation steps (Algorithm 1, line 17).

4.3 Trajectory Logging

Every online traversal produces a complete *trajectory*:

$$\mathcal{T} = \langle (v_1, e_1, o_1), (v_2, e_2, o_2), \dots, (v_n, e_n, o_n) \rangle, \quad (6)$$

recording each visited node v_i , the edge e_i taken, and the observation o_i returned by execution. The trajectory also records the final outcome: whether the traversal reached a successful outcome, timed out, or was redirected by a human operator. These trajectories are the raw input to ATR (Section 5).

5 PILLAR 3: ADAPTIVE TRAVERSAL REINFORCEMENT

While the offline pipeline (Section 3) provides a warm-start graph and the online engine (Section 4) traverses it, neither alone enables the graph to improve over time. The adaptive traversal reinforcement layer closes this gap by treating each online traversal as an agent whose path through the graph generates reinforcement signals.

5.1 Reinforcement Model

We augment the workflow graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with a reinforcement function $\phi : \mathcal{E} \cup \mathcal{V} \rightarrow \mathbb{R}^+$ that assigns a non-negative weight to each edge and each node. Initially, all edges and nodes receive a uniform reinforcement value ϕ_0 :

$$\phi(x) \leftarrow \phi_0, \quad \forall x \in \mathcal{E} \cup \mathcal{V}. \quad (7)$$

5.2 Reinforcement Deposition

After each online traversal producing trajectory $\mathcal{T}$, the system computes a *solution quality score* $Q(\mathcal{T}) \in [0, 1]$ based on the traversal outcome:

$$Q(\mathcal{T}) = w_{\text{useful}} \cdot \text{usefulness}(\mathcal{T}) + w_{\text{gnd}} \cdot \text{groundedness}(\mathcal{T}) + w_{\text{usr}} \cdot \text{user_score}(\mathcal{T}) \quad (8)$$

where w_{useful} , w_{gnd} , w_{usr} are configurable weights, $\text{usefulness}(\mathcal{T}) \in [0, 1]$ is the normalized usefulness score (judged by an LLM evaluator on a 1–5 scale) of the actions reached during the traversal, $\text{groundedness}(\mathcal{T})$ measures the fraction of conclusions supported by executed evidence, and $\text{user_score}(\mathcal{T})$ captures explicit user feedback (thumbs-up/down, if any), which is set to dominate the other weights.

For each edge e_i and node v_i on the traversal path, reinforcement is deposited only when the quality score exceeds a threshold δ_q (set to 4/5 in practice, ensuring only high-quality trajectories trigger updates):

$$\Delta\phi(x) = \begin{cases} \frac{Q(\mathcal{T})}{|\mathcal{T}|}, & \text{if } Q(\mathcal{T}) \geq \delta_q \\ 0, & \text{otherwise} \end{cases}, \quad \forall x \in \{e_i, v_i\} \subseteq \mathcal{T} \quad (9)$$

Shorter successful paths receive stronger per-element reinforcement, biasing the graph toward efficient diagnostic sequences. Node reinforcement captures the intrinsic value of individual actions (independent of which edges led to them), while edge reinforcement captures the value of action *transitions*.

5.3 Temporal Decay

To prevent early reinforcement patterns from dominating indefinitely and to allow the graph to adapt to changing infrastructure, all edge and node weights are decayed at regular intervals (epochs):

$$\phi(x) \leftarrow (1 - \rho) \cdot \phi(x), \quad \forall x \in \mathcal{E} \cup \mathcal{V}, \quad (10)$$

where $\rho \in (0, 1)$ is the decay rate. This serves three critical purposes:

- (1) **Staleness removal:** Paths that were effective for past failure modes but are no longer relevant (e.g., due to infrastructure upgrades) gradually lose their reinforcement.
- (2) **Exploration maintenance:** Without decay, the graph would converge prematurely to a narrow set of paths, losing the ability to explore alternatives for novel failure modes.
- (3) **Adaptivity:** The effective memory horizon of the system is controlled by ρ ; higher values yield faster adaptation but less stability.

5.4 Reinforcement-Weighted Subgraph Retrieval

During online traversal (Section 4), the Graph Loader uses edge reinforcement weights to probabilistically expand the subgraph.

At each node u , outgoing edges are sampled without replacement from:

$$p(e) = \frac{[\log(1 + \phi(e))]^\alpha}{\sum_{e' \in \mathcal{N}(u)} [\log(1 + \phi(e'))]^\alpha}, \quad (11)$$

where $\mathcal{N}(u)$ denotes the outgoing edges from u . The log transformation compresses the heavy-tailed weight distribution, and the concentration parameter α interpolates between exploitation ($\alpha \rightarrow \infty$, deterministic top- K) and exploration ($\alpha \rightarrow 0$, uniform).

Node weights. Node reinforcement weights $\phi(v)$ are included as properties in the context provided to the Action Planner, serving as soft priors that the LLM can override based on investigation context.

Edge synthesis. Beyond classical ACO, the system can *create new edges*: when a successful traversal consecutively visits unlinked nodes v_i and v_j , a new LEADS_TO edge is synthesized with an initial weight below the mean, requiring confirmation by subsequent traversals. These edges capture cross-incident action sequences discovered during online use.

5.5 Extensibility and Open Directions

A key advantage of the ATR framework is its simplicity: the reinforcement and decay rules are lightweight to implement yet open a rich design space for future refinement. We highlight three directions.

Learning from low-quality trajectories. The current design only deposits reinforcement for trajectories exceeding the quality threshold δ_q , discarding signals from lower-quality traversals. Applying *negative reinforcement* to suppress edges along failed trajectories could accelerate convergence, though careful design is needed to avoid permanently eliminating viable paths that failed due to transient conditions.

Scaling retrieval beyond exploitation. The current probability-based edge selection (Eq. 11) favors exploitation of proven paths. As graphs grow to larger scale, the retrieval mechanism will need more sophisticated dynamic pruning strategies that trim low-value nodes without eliminating infrequently used but genuinely useful actions.

Domain-differentiated reinforcement. Inspired by how ants from different colonies carry distinct chemical signatures for nest-mate recognition [9, 34], a natural extension is *domain-differentiated reinforcement*: maintaining separate weight channels $w_d(e)$ per problem domain d , so that traversals from capacity incidents reinforce different edges than those from authentication failures, reducing cross-domain noise and enabling faster convergence of domain-specific diagnostic strategies.

6 EVALUATION

We evaluate GraphMind in three settings: (i) offline extraction quality (Section 6.1); (ii) controlled online evaluation on held-out incidents (Section 6.2); and (iii) ATR impact on graph evolution and traversal (Section 6.3). Section 7 reports a production field study. These settings use different incident pools due to telemetry-retention constraints.

Table 2: Extraction performance: node-level (left) and edge-level by relationship type (right).

Type	Prec.	Rec.	F1	Edge Type	Prec.	Rec.	F1
Problem Nodes	0.98	0.92	0.95	CAUSES	1.00	0.95	0.98
Action Nodes	0.96	0.90	0.92	RESOLVES	0.86	0.86	0.86
				LEADS_TO	0.90	0.93	0.91
Overall	0.93	0.85	0.89	Overall	0.91	0.92	0.92

6.1 Offline Graph Construction Evaluation

To evaluate the accuracy of the offline graph construction pipeline, we manually annotated 30 incident tickets in which we invite engineers to annotate resolution steps manually. We annotated 71 problem nodes, 191 action nodes, and 262 edges in total from these human resolution traces to serve as ground truth.

Extraction Quality. Table 2 summarize the results against human annotations. The pipeline achieves node-level F1=0.89 (93% precision, 85% recall) and edge-level F1=0.92 (91% precision, 92% recall). Problem nodes reach 98% precision, reflecting accurate root-cause identification. Among edge types, CAUSES achieves the highest accuracy (F1=0.98) as causal connections are typically stated explicitly in incident documentation. The LEADS_TO relationship achieves moderate performance (F1=0.91), with the primary challenge being implicit action ordering. The RESOLVES relationship is most challenging (F1=0.86), reflecting the difficulty of linking specific actions to mitigation outcomes when telemetry quality is poor or resolution results from a combination of actions rather than a single step.

Error Analysis. We manually analyzed extraction errors and identified four dominant failure patterns. **Implicit actions** account for 38% of errors, where actions described indirectly (e.g., “after checking the firewall rules, the issue was resolved”) are not extracted as explicit action nodes. **Domain jargon** causes 26% of errors, as highly specialized terminology or abbreviations unknown to the base LLM occasionally lead to misclassification. **Cross-reference resolution** contributes 21%, where references to external documents or previous incidents are not always resolved correctly. The remaining 15% stem from **multimodal fusion** failures, where critical information split across text and images is not properly integrated.

Cost and Latency. Using GPT-4o pricing, the average cost is \$0.22 per incident (15.7 LLM calls, ~54K tokens). The median per-incident latency is 1.3 minutes (mean 2.9 min). Processing is trivially parallelizable, with Action Extraction dominating token usage due to reasoning-chain complexity.

Clustering Analysis. To evaluate clustering quality, we implemented an LLM-based cluster analysis pipeline that examines each cluster produced by agglomerative clustering, assessing whether grouped nodes are truly semantically equivalent. This LLM-based refinement generates improved cluster labels and identifies mismerged nodes, and we use it as one of three benchmark pipeline variants. We study clustering thresholds from 0.01 to 0.4 across three

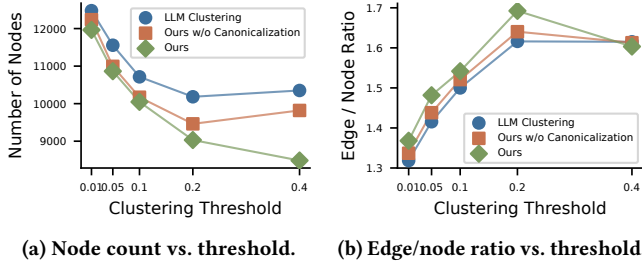


Figure 3: Graph size under varying clustering thresholds for three pipeline variants: Ours (canonicalization + pre-trained embeddings), Ours w/o Canonicalization, and LLM Clustering.

variants: *Ours* (canonicalization + embedding), *Ours w/o canonicalization* (embedding only), and *LLM Clustering* (LLM-based cluster-label refinement). Higher thresholds merge more semantically similar nodes, reducing graph size (from ~12K to ~8.5–10K nodes for one year’s traces) while increasing the edge-to-node ratio from 1.3 to 1.6, yielding a denser, more information-rich graph (Figure 3). Our canonicalization-based variant achieves comparable node reduction to the LLM Clustering variant at significantly lower cost and latency, making it the preferred choice for production deployment.

6.2 Online Traversal Evaluation

We evaluate GraphMind’s online agentic traversal using the full workflow graph constructed from one year of human resolution traces spanning approximately 1,300 production incidents, with roughly 12,000 active nodes at clustering threshold $\tau=0.01$. Each test incident is investigated by GraphMind autonomously without human intervention: given only the incident description, the system traverses the workflow graph, executes Kusto Query Language (KQL) diagnostic queries indicated in the action nodes to gather telemetry data, and attempts to reach a mitigation. Throughout the evaluation, a *traversal* denotes one autonomous investigation run for an incident; a *trajectory* is the logged sequence of graph nodes, edges, actions, and observations generated during that traversal; and a *conversation* refers to a real user interaction in the production deployment (Section 7). We organize metrics in a hierarchy of decreasing importance:

- **Mitigation Reach:** Whether the system reaches a mitigation action (binary).
- **Groundedness:** Whether conclusions are supported by executed diagnostic queries (binary).
- **Usefulness:** LLM-judged score (1–5) comparing the system’s recommended actions against human resolution logs.
- **Avg. KQL Succ.:** Number of successfully executed KQL queries per incident.
- **KQL Tbl. Recall:** Percentage of Kusto tables in human-logged queries also covered by agent-generated queries.
- **Action Recall:** Percentage of human-logged actions also performed by the system.

Note that Action Recall and KQL Table Recall are computed against incident human resolutions, that do not necessarily represent optimal diagnostic paths. We define hallucination rate as $1 - \text{groundedness}$, where groundedness indicates whether conclusions are supported by executed diagnostic evidence.

In-Sample Generalization. Before comparing against baselines, we verify that the graph generalizes beyond memorization. By tracking action provenance across 118 in-sample incidents, only 28% of cases retrieve actions originating from the *same* incident; the remaining 72% draw entirely from other incidents. Even without same-incident actions, performance remains strong: mitigation reach 98.8% vs. 97.0%, groundedness 92.9% vs. 87.9%, and usefulness 3.46 vs. 3.24, confirming that the workflow graph captures transferable diagnostic knowledge rather than incident-specific traces.

GraphMind vs. Trace-RAG. We compare **GraphMind** against a **Trace-RAG** baseline that retrieves LLM-condensed summaries of past incident traces via similarity search and uses them to guide an LLM agent, similar to **RCACopilot** [6]. Both systems benefit from LLM-based preprocessing; the key difference is knowledge representation: Trace-RAG retrieves per-incident summaries as flat documents, whereas GraphMind retrieves from a deduplicated workflow graph with explicit causal and procedural edges. The comparison therefore primarily tests graph-structured traversal vs. flat retrieval, though GraphMind’s graph also provides cross-incident deduplication and reinforcement-weighted prioritization that per-incident summaries lack.

Two domain experts conducted blind reviews of 19 matched incidents (the maximum feasible given expert availability). **GraphMind** scored 4.95/5 (94.7% perfect) vs. 3.68 for **Trace-RAG** (26.3% perfect). While the sample is small, the effect size is large (Cohen’s $d > 2$) and consistent: experts noted that **Trace-RAG** frequently produced incorrect query predicates and incomplete investigations.

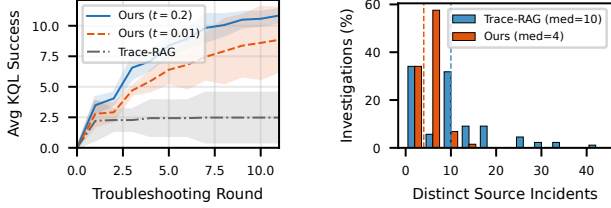
Table 3 reports automated metrics on 93 held-out incidents². **GraphMind** outperforms on mitigation reach (98.5% vs. 90.9%), groundedness (96.2% vs. 77.3%), and usefulness (3.49 vs. 3.32), with 4× higher KQL throughput (10.0 vs. 2.5; Figure 4a). **Trace-RAG**’s higher KQL table recall (0.925 vs. 0.852) is a structural artifact: it retrieves similar historical incidents and mimics their logged queries, whereas GraphMind explores diagnostically useful but potentially novel tables. Figure 4b shows **Trace-RAG** references a median of 10 source incidents vs. only 4 for **GraphMind**, yet achieves lower quality: per-incident summaries still carry redundant context across similar incidents, whereas the graph’s deduplicated structure surfaces only distinct, causally linked actions.

GraphMind vs. Raw-Trace RAG. Raw incident logs average ~5K tokens (P95: ~20K; max: ~39K). With Trace-RAG retrieving a median of 10 source incidents—and 28% retrieving more than 10—raw retrieval would consume ~50K tokens on average (39% of a 128K context window), with worst cases exceeding 234K tokens. Even when raw logs fit within the context window, they contain substantial noise (automated enrichment messages, status updates, formatting

²The 98.5% mitigation reach reported here and in the abstract is from this 93-incident evaluation. The 92–93% in Table 4 reflects a different set of 163 incidents: the ATR ablation (Section 6.3) was conducted at a later time, and the available incidents differ because each evaluation counts back from its run date over the telemetry data retention window.

Table 3: Comparison of GraphMind against the Trace-RAG. Values are mean $\pm$ 95% CI across pooled ticket observations. Best in bold.

Method	Mit. Reach	Ground.	Useful.	KQL Succ.	KQL Tbl. Rec.	Act. Rec.
Trace-RAG	90.9 $\pm$ 6.1%	77.3 $\pm$ 8.9%	3.32 $\pm$ 0.20	2.5 $\pm$ 0.4	0.925 $\pm$ 0.065	0.269 $\pm$ 0.031
GraphMind ($\tau=0.01$)	98.5 $\pm$ 2.1%	96.2 $\pm$ 3.3%	3.49 $\pm$ 0.12	10.0 $\pm$ 1.4	0.852 $\pm$ 0.078	0.239 $\pm$ 0.023



(a) Cumul. KQL success.

(b) Source incident dist.

Figure 4: Online comparison with Trace-RAG. (a) Cumulative KQL query success over troubleshooting rounds; GraphMind sustains high throughput while Trace-RAG plateaus. (b) Distribution of distinct source incidents referenced per investigation; GraphMind draws from fewer incidents (median 4 vs. 10) yet achieves higher diagnostic quality.

artifacts) that degrades signal-to-noise ratio and increases hallucination risk. This motivates both Trace-RAG’s LLM-condensed summaries and GraphMind’s graph-based deduplication as necessary compression, with GraphMind achieving further savings through cross-incident node sharing.

Native vs. GHC Orchestration. We compare GraphMind’s **native** orchestrator (Section 4) against **GHC** (GitHub Copilot [16]), a general-purpose agentic framework that accesses the same graph and tools via Model Context Protocol (MCP) [1]. **GHC** achieves highest action recall (0.297) but only 58.0% groundedness and 2.4 KQL queries per incident vs. the **native** orchestrator’s 10.0–11.5. GHC adopts a plan-once-then-execute strategy rather than iterative explore-exploit-replan cycles, yielding shallower investigations despite spending its budget on reasoning (8.1 steps/ticket) and tool discovery (2.6 skill lookups).

Sensitivity Analysis: Retrieval Parameters. We study the impact of (k_p, k_a) on online performance across a 6×6 grid with $k_p, k_a \in \{1, 3, 5, 7, 10, 15\}$ (3 runs each; Figure 5). Increasing k_a improves action recall and KQL throughput, but $k_a=1$ is significantly worse across all metrics, confirming that action diversity is critical. The k_p parameter shows a concave pattern: moderate values ($k_p \in \{3, 5\}$) yield the best mitigation reach and groundedness, while extremes underperform. The $(k_p=3, k_a=7)$ region achieves the best balance.

6.3 Adaptive Traversal Reinforcement Evaluation

We evaluate ATR along two dimensions: graph evolution behavior and impact on online traversal.

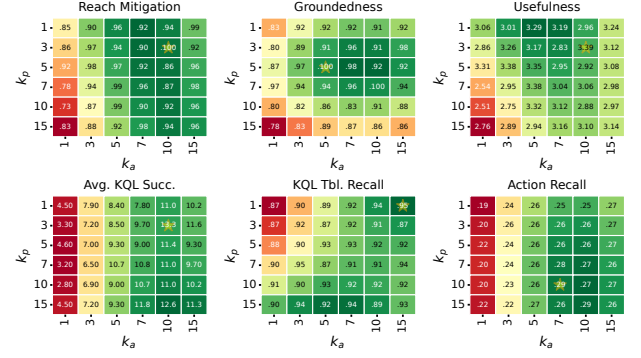
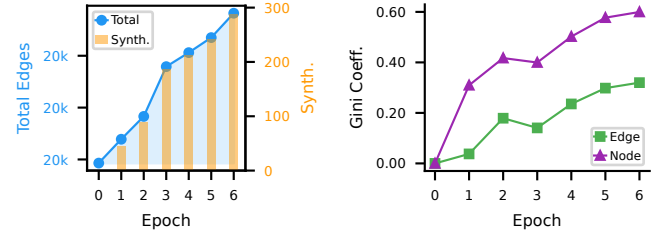


Figure 5: Impact of retrieval parameters (k_p, k_a) on online troubleshooting metrics. Best cell marked with *.



(a) Cumul. edge synthesis.

(b) Gini coefficient.

Figure 6: Reinforcement evolution over six epochs. (a) 289 synthesized edges grow the graph as traversals discover cross-incident sequences. (b) Gini coefficient increases steadily, indicating weight concentration on successful paths.

Graph Evolution. We evolve the workflow graph over six reinforcement epochs (15 traversal trajectories each). We use the simplest configuration: decay $\rho=0$ (retaining all reinforcement) and concentration $\alpha=0$ (uniform edge selection), isolating the effect of reinforcement deposition and edge synthesis³. The graph begins with 13,344 nodes and 19,893 edges. After six epochs, 289 new edges are synthesized as traversals discover cross-incident action sequences, growing the total to 20,182. Figure 6(a) shows cumulative edge synthesis growth; Figure 6(b) tracks the Gini coefficient [15] of reinforcement weights: node Gini rises to 0.60 and edge Gini to 0.32, indicating concentration on frequently successful paths while maintaining a long tail of alternatives.

Figure 7 visualizes the reinforced subgraph at each epoch. Distinct “highway” clusters emerge for common failure categories (e.g., capacity, connectivity, authentication), connected by synthesized edges that bridge incident types and enable cross-domain diagnostic transfer.

Impact on Online Traversal. We compare *With Reinforcement* (learned weights ϕ bias edge selection via Eq. 11) against *Without Reinforcement* (uniform weights) on 163 recent incidents across 8 runs (4 per condition; $n=656$ and $n=674$ samples). Evaluation incidents may

³At the current graph scale, retrieved subgraphs fit within the 1M-token context window; reinforcement-based trimming would further improve token efficiency.

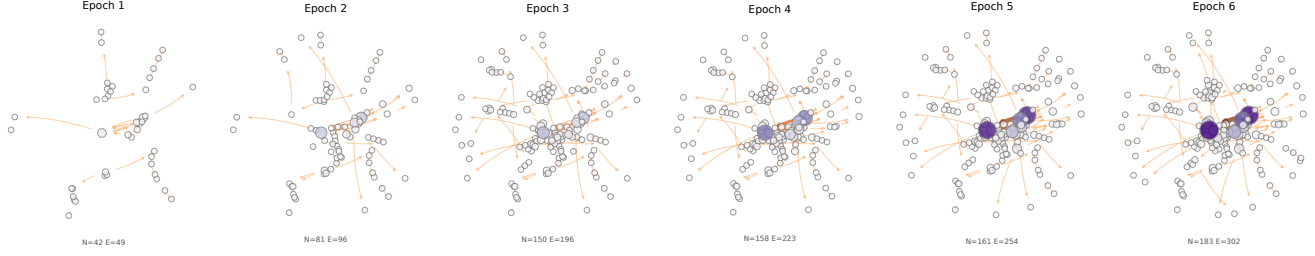


Figure 7: Reinforced subgraph evolution across six epochs. Node size and edge thickness are proportional to reinforcement weight. The subgraph grows from 42 nodes / 49 edges (Epoch 1) to 183 nodes / 302 edges (Epoch 6), with “highway” clusters emerging as traversal experience accumulates.

Table 4: Impact of adaptive traversal reinforcement on online performance. Values are mean $\pm$ 95% CI over 8 runs (4 per condition). Best in bold.

Condition	Mit. Reach	Ground.	Useful.	KQL Succ.	KQL Tbl. Rec.	Act. Rec.
W/o Reinf. ($n=674$)	92.0 $\pm$ 2.1%	89.5 $\pm$ 2.3%	3.331 $\pm$ 0.060	15.1 $\pm$ 1.1	0.792 $\pm$ 0.026	0.218 $\pm$ 0.009
W/ Reinf. ($n=656$)	93.3 $\pm$ 1.9%	92.2 $\pm$ 2.1%	3.366 $\pm$ 0.061	17.2 $\pm$ 1.2	0.767 $\pm$ 0.029	0.228 $\pm$ 0.009

involve different problem categories than those encountered during deposition, testing generalization.

Table 4 shows that reinforcement improves groundedness by 2.7 percentage points (92.2% vs. 89.5%) and mitigation reach by 1.3 points (93.3% vs. 92.0%), while KQL throughput increases from 15.1 to 17.2 and action recall from 0.218 to 0.228. Although confidence intervals overlap for some individual metrics, improvements are *directionally consistent* across all six metrics and all four independent runs per condition, indicating a robust signal rather than statistical noise. The practical significance is notable: in a production setting processing thousands of incidents, a 2.7-point groundedness gain translates to hundreds of additional evidence-backed diagnoses. The KQL throughput gap (15.1 vs. 17.2, non-overlapping CIs) is the strongest individual signal, demonstrating that reinforced paths guide agents toward more productive diagnostic queries.

7 DEPLOYMENT AND USAGE STUDY

We report a field study of GraphMind’s production deployment across four cloud database services at Microsoft, covering 88 real-world investigation conversations over a 35-day period on two orchestrators: the native orchestrator (62 conversations) and GHC (26 conversations). All conversations were initiated by on-call engineers during live incident triage. The 62 native-orchestrator conversations were independently scored on a 1–5 usefulness scale by the authors.

Adoption and Engagement. Usage grew substantially over the study period; 50 of 62 native conversations (81%) occurred in the final two weeks. Figure 8 summarizes the 62 native conversations. The median conversation spans 6 rounds (mean 7.4), with a right-skewed tail reaching 40 rounds for complex incidents. Median per-round latency is 7.7 s (P95: 25.1 s), remaining within interactive bounds despite multi-step graph retrieval, LLM planning, KQL execution, and result synthesis.

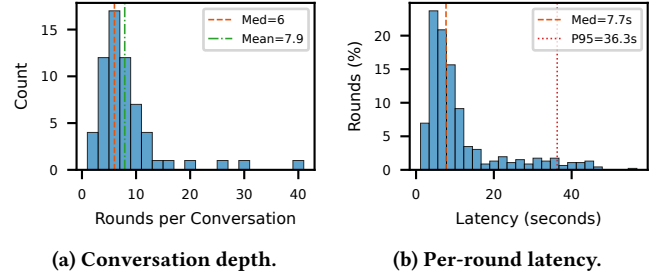


Figure 8: Engagement and responsiveness across 62 production conversations. (a) Multi-turn investigations (median 6 rounds). (b) Median latency 7.7 s; P95 dominated by complex telemetry queries.

Diagnostic Effectiveness and Outcomes. The system generated 784 KQL queries across 62 conversations, of which 659 (84%) returned data (median 8 per conversation). Manual outcome classification shows that 26% of conversations ($n=16$) reached a definitive root cause, 71% ($n=44$) produced useful diagnostic evidence advancing the investigation, and only 3% ($n=2$) yielded no actionable output. The mean usefulness score is 3.10/5, with 79% rated ≥ 3 and 34% rated 4–5. Low-scoring conversations (21%) typically involved incidents outside graph coverage or expired telemetry.

Failure Analysis. We analyzed the 21% of conversations scoring below 3 and the 3% ($n=2$) with no actionable output. Three dominant failure modes emerge: (1) *coverage gaps* (45% of failures): incidents involving recently deployed services or rare failure modes not yet represented in the graph; (2) *telemetry expiration* (35%): diagnostic queries returning empty results because retention windows had elapsed by investigation time; and (3) *cross-team dependencies* (20%): incidents requiring access to restricted systems or data owned by other teams. Notably, the ATR mechanism can partially address coverage gaps over time, as new traversals through previously uncovered regions synthesize edges and deposit reinforcement, expanding the graph’s effective coverage.

Summary. Across 88 conversations on two platforms, GraphMind achieves interactive latency (median 7.7 s), high query success (84%), and actionable output in 97% of detailed conversations, validating that controlled evaluation gains translate to real-world triage workflows.

8 DISCUSSION

From static knowledge to living infrastructure. The ATR layer transforms the workflow graph from a static artifact into a self-improving ecosystem. Each traversal contributes signal that shapes the graph's structure; over hundreds of traversals, emergent properties (reinforced highways, dormant dead ends) arise that no individual engineer could design manually, representing a form of *collective intelligence*.

Edge reinforcement as interpretability. Reinforcement values provide a human-interpretable summary of learned diagnostic strategies. Domain experts can inspect highest-reinforcement paths for validation and knowledge discovery; anomalously high reinforcement on unexpected edges can flag emergent failure patterns.

Connections to reinforcement learning. ATR can be viewed as *tabular reinforcement learning* where edge/node weights correspond to Q-values. The deposition rule (Eq. 9) is analogous to temporal-difference updates and decay (Eq. 10) provides a discount factor, suggesting extensions such as learned value functions or MCTS-based planning over the graph.

Limitations. The current reinforcement model assumes edge independence; extending to path-level reinforcement with contextual credit assignment is an important future direction. The decay rate ρ is fixed; adaptive decay based on detected distribution shifts could improve responsiveness. The evaluation uses a single baseline (Trace-RAG over LLM-condensed incident summaries); while this tests graph-structured traversal vs. flat retrieval with comparable preprocessing, both systems share LLM-based trace processing, making it a meaningful comparison of knowledge representation rather than extraction quality. Finally, the system is evaluated on a single domain (cloud database incident management); generalization to other operational domains remains to be validated.

9 CONCLUSIONS

We presented GraphMind, a system that transforms static workflow graphs into living, self-improving operational assets, drawing on Ant Colony Optimization as foundational inspiration. The system operates through three pillars: (1) an offline pipeline that warm-starts the graph from historical traces (node F1=0.89, edge F1=0.92 at \$0.22/ticket); (2) an online multi-agent traversal engine achieving 98.5% mitigation reach, 4 $\times$ higher successful KQL execution throughput, and 4.95/5 expert scores; and (3) Adaptive Traversal Reinforcement (ATR) that deposits reinforcement on successful paths, improving groundedness by 2.7 percentage points and KQL throughput by 14% over uniform traversal across 8 independent runs, with directionally consistent gains across all metrics. In a 35-day production field study across four database services, 97% of conversations produce actionable output, with failure cases primarily attributable to coverage gaps and telemetry expiration rather than system design flaws.

Several directions remain open. Additional ablations isolating individual graph components (e.g., edge structure, deduplication, reinforcement) would further clarify their contributions. The edge-independent reinforcement model could be extended to path-level reinforcement. Adaptive decay rates and domain-differentiated reinforcement channels could improve responsiveness. Finally, the

swarm-inspired paradigm may generalize beyond incident management to software debugging, clinical decision support, and supply chain optimization.

10 GENAI USAGE DISCLOSURE

ChatGPT and GitHub Copilot were utilized to assist in generating sections of this work, including text, tables, graphs, code, data, and citations.

REFERENCES

- [1] Anthropic. 2024. Model Context Protocol (MCP). <https://modelcontextprotocol.io/>. Open standard for connecting AI applications to external systems; enables structured, secure interaction between LLMs and data sources/tools via a universal protocol.
- [2] Anthropic. 2025. Claude Code. <https://docs.anthropic.com/en/docs/claude-code>. Agentic coding tool with built-in MCP client support.
- [3] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [4] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. 2013. Translating Embeddings for Modeling Multi-relational Data. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [5] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [6] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, Jun Zeng, Supriyo Ghosh, Xuchao Zhang, Chaoyun Zhang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Tianyin Xu. 2024. Automatic Root Cause Analysis via Large Language Models for Cloud Incidents. In *Proceedings of the 19th European Conference on Computer Systems (EuroSys)*, 674–688. <https://doi.org/10.1145/3627703.3629553>
- [7] Lara N. DeLong, Ramon Fernandez Mir, and Jacques D. Fleuriot. 2024. Neurosymbolic AI for Reasoning Over Knowledge Graphs: A Survey. *IEEE Transactions on Neural Networks and Learning Systems* (2024). <https://doi.org/10.1109/TNNLS.2024.3420218>
- [8] Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. 2018. Convolutional 2D Knowledge Graph Embeddings. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [9] Patrizia d'Ettorre and Alain Lenoir. 2010. Nestmate Recognition. In *Ant Ecology*, Lori Lach, Catherine L. Parr, and Kirsti L. Abbott (Eds.). Oxford University Press, 194–209.
- [10] Gianni Di Caro and Marco Dorigo. 1998. AntNet: distributed stigmergetic control for communications networks. *Journal of Artificial Intelligence Research* 9 (1998), 317–365.
- [11] Marco Dorigo and Luca Maria Gambardella. 1997. Ant colony system: a cooperative learning approach to the traveling salesman problem. In *IEEE Transactions on Evolutionary Computation*, Vol. 1, 53–66. <https://doi.org/10.1109/4235.585892>
- [12] Marco Dorigo, Vittorio Maniezzo, and Alberto Colomi. 1996. Ant system: optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)* 26, 1 (1996), 29–41. <https://doi.org/10.1109/3477.484436>
- [13] Marco Dorigo and Thomas Stützle. 2004. *Ant Colony Optimization*. MIT Press.
- [14] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitan, Robert Osazuwa Ness, and Jonathan Larson. 2025. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv:2404.16130 [cs.CL]* <https://arxiv.org/abs/2404.16130>
- [15] Corrado Gini. 1921. Measurement of inequality of incomes. *The economic journal* 31, 121 (1921), 124–125.
- [16] GitHub. 2026. GitHub Copilot CLI. <https://github.blog/changelog/2026-02-25-github-copilot-cli-is-now-generally-available/>. MCP-compatible agentic coding assistant for the command line.
- [17] Haoyu Han, Yu Wang, Harry Shomer, Kai Guo, Jiayuan Ding, Yongjia Lei, Mahantesh Halappanavar, Ryan A. Rossi, Subhabrata Mukherjee, et al. 2025. Retrieval-Augmented Generation with Graphs (GraphRAG). *arXiv preprint arXiv:2501.00309* (2025).
- [18] Stephen C. Johnson. 1967. Hierarchical Clustering Schemes. *Psychometrika* 32, 3 (Sept. 1967), 241–254. <https://doi.org/10.1007/BF02289588>
- [19] Liubov Kovrigina, Irina Toma, et al. 2024. LLM-based SPARQL Query Generation from Natural Language over Federated Knowledge Graphs. In *Proceedings of the International Semantic Web Conference (ISWC)*. <https://arxiv.org/abs/2410.06062>
- [20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al.

2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [21] Daniel Merkle, Martin Middendorf, and Hartmut Schmeck. 2002. Ant colony optimization for resource-constrained project scheduling. In *IEEE Transactions on Evolutionary Computation*, Vol. 6. 333–346.
- [22] OpenAI. 2023. Function Calling. <https://platform.openai.com/docs/guides/function-calling>. Accessed: 2025-05-15.
- [23] OpenAI. 2023. GPT-4 Technical Report. <https://cdn.openai.com/papers/gpt-4.pdf>.
- [24] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2024. Unifying Large Language Models and Knowledge Graphs: A Roadmap. *IEEE Transactions on Knowledge and Data Engineering* 36, 7 (2024), 3580–3601. <https://doi.org/10.1109/TKDE.2024.3352100>
- [25] Kartik Ravichandran, Namrata Gurumurthy Kumar, Prateek Mishra, and Rahul Agrawal. 2025. OG-RAG: Ontology-Grounded Retrieval-Augmented Generation for Large Language Models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. <https://doi.org/10.18653/v1/2025.emnlp-main.1674>
- [26] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [27] Mili Shah, Joyce Cahoon, Mirco Milletari, Jing Tian, Fotis Psallidas, Andreas Mueller, and Nick Litombe. 2024. Improving LLM-based KGQA for multi-hop Question Answering with implicit reasoning in few-shot examples. In *Proceedings of the 1st Workshop on Knowledge Graphs and Large Language Models (KaLLM 2024)*. Association for Computational Linguistics, Bangkok, Thailand, 125–135. <https://doi.org/10.18653/v1/2024.kallm-1.13>
- [28] Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. 2025. Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG. *arXiv preprint arXiv:2501.09136* (2025).
- [29] Vikramank Singh, Kapil Eknath Vaidya, Vinayshekhar Bannihatti Kumar, Sopan Khosla, Murali Narayanaswamy, Rashmi Gangadharaiah, and Tim Kraska. 2024. Panda: Performance Debugging for Databases using LLM Agents. In *Proceedings of the Conference on Innovative Data Systems Research (CIDR)*.
- [30] Thomas Stützle and Holger H. Hoos. 2000. MAX-MIN Ant System. *Future Generation Computer Systems* 16, 8 (2000), 889–914. [https://doi.org/10.1016/S0167-739X\(00\)00043-1](https://doi.org/10.1016/S0167-739X(00)00043-1)
- [31] Sina Tabakhi, Parham Moradi, and Fardin Akhlaghian. 2014. An unsupervised feature selection algorithm based on ant colony optimization. *Engineering Applications of Artificial Intelligence* 32 (2014), 112–123.
- [32] Wil M. P. van der Aalst. 2011. *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Springer. <https://doi.org/10.1007/978-3-642-19345-3>
- [33] Wil M. P. van der Aalst, Ton Weijters, and Laura Maruster. 2004. Workflow Mining: Discovering Process Models from Event Logs. *IEEE Transactions on Knowledge and Data Engineering* 16, 9 (2004), 1128–1142. <https://doi.org/10.1109/TKDE.2004.47>
- [34] Jelle S. van Zweden and Patrizia d’Ettorre. 2010. Nestmate Recognition in Social Insects and the Role of Hydrocarbons. In *Insect Hydrocarbons: Biology, Biochemistry, and Chemical Ecology*, Gary J. Blomquist and Anne-Geneviève Bagnères (Eds.). Cambridge University Press, 222–243.
- [35] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345. <https://doi.org/10.1007/s11704-024-40231-1>
- [36] Zhen Wang, Jianwen Zhang, Jianlin Feng, and Zheng Chen. 2014. Knowledge Graph Embedding by Translating on Hyperplanes. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [37] Xinyi Xia, Yijun Zhu, Jianan Guo, et al. 2025. Knowledge Graph Finetuning Enhances Knowledge Manipulation in Large Language Models. In *Proceedings of the International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=oMFOKjwaRS>
- [38] Yuqing Xue, Zhuoran Wang, Wei Sun, Fanxin Meng, Wenchao Zhang, Zhenyu Li, et al. 2025. TRIANGLE: A Benchmark and Framework for Automated Incident Triage in Large-Scale Cloud Systems. In *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. https://netman.aiops.org/wp-content/uploads/2025/10/TRIANGLE_FSE25.pdf.
- [39] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- [40] William Zhang, Yiwen Zhu, Yunlei Lu, Mathieu Demarne, Wenjing Wang, Kai Deng, Nutan Sahoo, Katherine Lin, Miso Cilimdžić, and Subru Krishnan. 2025. FLAIR: Feedback Learning for Adaptive Information Retrieval. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25)*. Association for Computing Machinery, Seoul, Republic of Korea. <https://doi.org/10.1145/3746252.3761553> Work done while interning at Microsoft.
- [41] Xuanhe Zhou, Guoliang Li, Zhaoyan Liu, et al. 2024. D-Bot: Database Diagnosis System using Large Language Models. *Proceedings of the VLDB Endowment* 17, 10 (2024), 2514–2527. <https://doi.org/10.14778/3675034.3675043>
- [42] Yiwen Zhu, Mathieu Demarne, Kai Deng, Wenjing Wang, Nutan Sahoo, Divya Vermareddy, Hannah Lerner, Yunlei Lu, Swati Bararia, Anjali Bhavan, William Zhang, Xia Li, Katherine Lin, Miso Cilimdžić, and Subru Krishnan. 2025. DECO: Life-Cycle Management of Enterprise-Grade Copilots. <https://doi.org/10.1145/3770854.3783949> arXiv:2412.06099 [cs.SE]